

MySQL + Node.js CRUD

Connect · Insert · Update · Delete · Select

What You Will Learn

- Connect Node.js to a MySQL database
- Insert, Update, Delete, and Select/Show data using Node.js

Pre-Requirements

Make sure you have these installed on your computer:

- Node.js — nodejs.org
- XAMPP (for MySQL server) — [apachefriends.org](https://www.apachefriends.org)

STEP
1

Create the Project Folder

Open your terminal and run:

```
Create folder: MYSQL_Node_CRUD  
And open in VS Code
```

Then initialize a Node.js project:

```
npm init -y
```

This creates a package.json file in your folder.

STEP
2

Install MySQL Driver

```
npm install mysql2
```

This installs the mysql2 package so Node.js can talk to MySQL.

STEP
3

Create the Database and Table

- Open XAMPP → Start Apache and MySQL
- Open your browser → go to <http://localhost/phpmyadmin>

- Click New → Create a database named users
- Select the users database → Go to SQL tab → Run this query:

```
CREATE TABLE tbl_users (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100),  
  email VARCHAR(100)  
);
```

STEP
4

Create db.js File

Create a file named db.js in your project folder and add this code:

```
const mysql = require('mysql2');  
  
const connection = mysql.createConnection({  
  host: 'localhost',  
  user: 'root',  
  password: '', // leave empty if no password set in XAMPP  
  database: 'users'  
});  
  
connection.connect((err) => {  
  err ?  
    hasError(err) :  
    console.log('Connected to MySQL..!');  
});  
  
function hasError(err) {  
  if (err) {  
    console.error('Error:', err);  
    return;  
  }  
}
```

STEP
5

INSERT a Single User

Add this code in db.js after the connect block:

```
const sql = 'INSERT INTO tbl_users (name, email) VALUES (?, ?)';
const values = ['Meet', 'meet@gmail.com'];

connection.query(sql, values, (err, result) => {
  err ?
    hasError(err) :
    console.log('Inserted ID:', result.insertId);
});
```

Run the file:

```
node db.js
```

STEP 6

INSERT Multiple Users at Once

```
const usersData = [
  ['Jeet', 'jeet@gmail.com'],
  ['Rahul', 'rahul@gmail.com'],
  ['Akash', 'akash@gmail.com'],
];

const insert_multi_sql = 'INSERT INTO tbl_users (name, email) VALUES ?';

connection.query(insert_multi_sql, [usersData], (err, result) => {
  err ?
    hasError(err) :
    console.log('Users Inserted', result.insertId);
});
```

STEP 7

UPDATE a User

```
const update_sql =
  'UPDATE tbl_users SET name="Jeetss" WHERE email="jeet@gmail.com"';

connection.query(update_sql, (err, result) => {
  err ?
    hasError(err) :
    console.log('Users Updated', result.insertId);
});
```

STEP 8

DELETE a User

```
const delete_sql =
  'DELETE FROM tbl_users WHERE name="Jeetss"';

connection.query(delete_sql, (err) => {
  err ?
    hasError(err) :
    console.log('User Deleted..!');
});
```

STEP
9

SELECT / Show All Users

```
connection.query('SELECT * FROM tbl_users', (err, results) => {
  err ?
    hasError(err) :
    console.log('All Users:', results);
});
```

STEP
10

Close the Connection

Always close the connection at the end of your file:

```
connection.end();
```

How to Run the File

```
node db.js
```

connection.query() with ?

=> mysql2 replaces the ? with values on the Node.js side before sending

=> MySQL receives one complete SQL string like:

```
INSERT INTO tbl_users (name, email) VALUES ('Meet', 'meet@gmail.com')
```

=> MySQL just runs it directly, no preparation

=> No protection against SQL injection if you build SQL manually

connection.execute() with ?

=> SQL and values are sent to MySQL separately

=> MySQL compiles the SQL first, then fills in the ? values safely

=> MySQL receives:

```
INSERT INTO tbl_users (name, email) VALUES (?, ?)    -- step 1: SQL sent first
('Meet', 'meet@gmail.com')                          -- step 2: values sent after
```

=> MySQL compiles the SQL once, then safely fills in the values

=> Built-in SQL injection protection

Quick Summary Table

Step	What it does	SQL Keyword
5	Add one user	INSERT
6	Add many users	INSERT
7	Change user data	UPDATE
8	Remove a user	DELETE
9	Show all users	SELECT

Common Errors & Fixes

Error	Fix
ER_ACCESS_DENIED_ERROR	Check your MySQL username/password in db.js
ER_BAD_DB_ERROR	Make sure the database users is created in phpMyAdmin
Cannot find module 'mysql2'	Run npm install mysql2 again

Run node db.js after every step to see the output in the terminal!